

TARJETA DE BOLSILLO — GIT Y GITHUB

Todos los comandos y acciones del módulo CSTI12002, agrupados por momento de uso · Imprimir y tener a mano

Uso	Referencia rápida permanente. Los comandos se ejecutan en la terminal integrada de VS Code (Ctrl + ` / Ctrl + ñ). Lo que va entre < > se reemplaza por el valor propio, sin los signos.
Regla de oro	Ante la duda, SIEMPRE: git status primero. Nunca se borra la carpeta .git ni se trabaja sin hacer git pull antes.

0. Diagnóstico (el comando que se usa antes que todos)

Comando	Qué hace / cuándo usarlo
git status	Muestra rama actual, archivos modificados, preparados y sin seguimiento. Úselo antes y después de cada operación.
git --version	Verifica que Git está instalado y su versión.

1. Configuración (una única vez por computadora)

Comando	Qué hace
git config --global user.name "Nombre Apellido"	Define el nombre que firma sus commits.
git config --global user.email "correo@dominio.com"	Define el correo (debe coincidir con el de GitHub).
git config --global init.defaultBranch main	La rama inicial de todo repositorio nuevo será main.
git config --global pull.rebase false	Estrategia de pull del curso: fusión clásica.
git config --list	Muestra toda la configuración vigente.

2. Crear u obtener un repositorio

Comando	Qué hace
git init	Convierte la carpeta actual en repositorio (crea .git). Se usa UNA vez por proyecto.
git clone <url>	Descarga un repositorio remoto COMPLETO (con todo su historial). Así se trae el repo de un compañero que lo invitó como colaborador.

3. Ciclo diario: preparar y confirmar

Comando	Qué hace
git add <archivo>	Prepara ESE archivo (lo pasa al staging).
git add .	Prepara TODOS los cambios de la carpeta actual. Usar con consciencia (revisar antes con git status).
git commit -m "tipo: descripción"	Confirma lo preparado como instantánea permanente. Tipos del curso: feat, fix, docs, style, chore. Mensaje imperativo, ≤ ~50 caracteres.
git commit --amend -m "mensaje"	Corrige el mensaje del ÚLTIMO commit (solo si aún no se hizo push).

4. Inspeccionar: historial y diferencias

Comando	Qué hace
git log	Historial completo: hash, autor, fecha, mensaje.
git log --oneline	Historial compacto: una línea por commit.
git log --oneline --all --graph	Historial gráfico de TODAS las ramas (dibuja las bifurcaciones).
git log --stat	Historial con los archivos tocados por cada commit.
git show <hash>	Detalle y diff de un commit específico (basta los 7 primeros caracteres del hash).
git diff	Cambios hechos y AÚN NO preparados (working vs staging).
git diff --staged	Cambios ya preparados vs el último commit (lo que entrará al próximo commit).

5. Deshacer (los salvavidas)

Comando	Qué hace
git restore <archivo>	DESCARTA los cambios locales del archivo y vuelve a la última versión confirmada. No hay vuelta atrás de lo descartado.
git restore --staged <archivo>	Saca el archivo del staging (deshace el git add) SIN perder los cambios.
git commit --amend	Rehace el último commit (mensaje u olvido de un archivo ya agregado con add). Solo antes del push.

6. Ramas (el motor del protocolo)

Comando	Qué hace
git branch	Lista las ramas locales; el * marca la actual.
git switch <rama>	Cambia a esa rama (los archivos del editor cambian con ella).
git switch -c practica/sNN-tema	CREA la rama y se cambia a ella en un paso. Convención del curso para cada práctica. (Equivale al clásico git checkout -b).
git branch -d <rama>	Borra una rama local YA fusionada (higiene tras el merge).

7. Fusionar y resolver conflictos

Comando / acción	Qué hace
git merge <rama>	Fusiona <rama> DENTRO de la rama actual (pararse primero en la rama destino, normalmente main).
Conflicto: <<<<<< HEAD ... ===== ... >>>>>>	Git pide decisión humana: editar el archivo dejando la versión final, BORRAR los marcadores, guardar.
Tras resolver: git add <archivo> + git commit	Sella la resolución del conflicto. En VS Code: botones Aceptar cambio actual / entrante / ambos.

8. Remotos y sincronización con GitHub

Comando	Qué hace
git remote add origin <url>	Conecta el repo local con el remoto de GitHub (una vez por proyecto).
git remote -v	Muestra los remotos configurados (verificación).
git remote set-url origin <url>	Corrige la URL del remoto (si se conectó mal).

git push -u origin main	PRIMERA publicación de la rama main (la -u recuerda el destino).
git push -u origin practica/sNN-tema	Publica la rama de la práctica (para abrir el Pull Request).
git push	Sube los commits locales (tras la primera vez con -u).
git pull	Baja Y fusiona las novedades del remoto. SIEMPRE antes de empezar a trabajar.
git fetch	Baja las novedades SIN fusionar (para revisarlas primero).

9. Acciones en la ventana de GitHub (sin comandos)

Acción	Dónde / cómo
Crear repositorio	Botón + → New repository → nombre → Public → SIN inicializar con README si el repo ya existe en local.
Abrir Pull Request	Banner "Compare & pull request" tras el push, o pestaña Pull requests → New → base: main ← compare: practica/...
Revisar un PR (coevaluación)	PR → Files changed → + en la línea → comentario → Review changes: Comment / Approve / Request changes.
Fusionar y limpiar	Merge pull request (Create a merge commit) → Delete branch. En local: git switch main + git pull.
Invitar colaborador	Settings → Collaborators → Add people (el par acepta desde su correo/notificaciones).
Ver quién aportó qué	Insights → Contributors · o Blame sobre el archivo.
Publicar el sitio (GitHub Pages)	Settings → Pages → Deploy from a branch → main / (root) → Save → URL: usuario.github.io/repositorio/...
Editar rápido en la web	Ícono lápiz sobre el archivo → commit desde el navegador (luego git pull en local).
Atajos de la web	t: buscar archivo · . : abrir github.dev (VS Code en el navegador) · y: permalink · ?: ver todos los atajos.

10. El protocolo del curso en una línea de comandos

```
git pull
git switch -c practica/sNN-tema
# ...trabajar... (commits atómicos: git add . && git commit -m "feat: ...")
git push -u origin practica/sNN-tema
# GitHub: Compare & pull request → describir → Merge → Delete branch
git switch main
git pull
git branch -d practica/sNN-tema
```

♦ Los 5 tipos de commit del curso: **feat** (contenido/funcionalidad nueva) · **fix** (corrección) · **docs** (README/bitácora) · **style** (presentación/CSS) · **chore** (estructura/mantenimiento). En evaluaciones: commits periódicos = evidencia de autoría, y la URL del repositorio va en repositorio_url.txt dentro del zip de entrega.